JOHANNES KEPLER
UNIVERSITY LINZ

Volodymyr Yelisieiev
12340334

May 29, 2026

NDSwin-JAX: Implementing and Evaluating an N-Dimensional Shifted Window Transformer in JAX

Institute for Machine Learning

Technical Report
Practical Work in AI (BSc)
Public repository:
https://github.com/volodymyr-yelisieiev/ndswin-jax

JOHANNES KEPLER
UNIVERSITY LINZ
Altenberger Straße 69
4040 Linz, Austria
jku.at

Contents

Abstract

This report presents *NDSwin-JAX*, a JAX/Flax implementation of the Swin Transformer that was refactored into a genuinely N-dimensional framework. The core engineering objective was to preserve the shifted-window design of the original Swin architecture while removing its hard dependency on 2D image grids. In practice, that meant rewriting patch embedding, window partitioning and reversal, cyclic shifting, relative position indexing, and hierarchical stage composition so that they operate on arbitrary spatial dimensionalities controlled only by configuration.

The final publishable artifact set evaluates the implementation on three workloads: 2D classification on CIFAR-10, 2D classification on CIFAR-100, and 3D voxel classification on a ModelNet40 export. The saved restored-best checkpoints reach validation top-1 accuracies of 0.9182 on CIFAR-10, 0.6948 on CIFAR-100, and 0.7421 on ModelNet40. Recomputed held-out test evaluations are also preserved for the final checkpoints, with test top-1 accuracies of 0.8995, 0.6963, and 0.6965, respectively.

These results support the central claim of the project: the implementation successfully generalizes shifted-window transformers beyond 2D and remains trainable on both image and voxel inputs under a shared software stack. The report stays conservative about scope. The evidence consists of single retained runs rather than repeated-seed experiments, and the 3D result uses a voxelized ModelNet40 export rather than a raw point-cloud protocol. Consequently, the contribution is best understood as a traceable implementation and empirical validation study rather than a state-of-the-art benchmark claim.

Story Summary

What is the central question? Can the shifted-window Swin Transformer be implemented as a reusable N-dimensional JAX/Flax framework and trained through one shared experiment pipeline on both 2D images and 3D voxel data?

Why is this question important? Many practical visual datasets are not limited to ordinary 2D images, but most Swin implementations and training utilities still contain 2D assumptions. A dimension-aware implementation makes the architecture easier to reuse for volumetric and higher-dimensional workloads.

What evidence/data are needed to answer this question? The evidence consists of code paths that operate on configurable spatial dimensionality, configuration files for 2D and 3D workloads, saved sweep summaries, restored-best checkpoint metrics, held-out test recomputations, logs, and exported model weights.

What methods are used to get this evidence/data? The project implements N-dimensional patch embedding, window partitioning, shifted attention, patch merging, training, sweeps, and queue execution in JAX/Flax. It evaluates the resulting system on CIFAR-10, CIFAR-100, and a voxelized ModelNet40 export.

What analyses must be applied to the data to answer the central question? The report checks whether the same implementation and command surface train successfully across the selected dimensionalities, whether validation-selected checkpoints generalize to held-out test splits, and whether the reported numbers can be traced back to committed artifacts.

What evidence/data values were obtained? The final restored checkpoints reach validation top-1 accuracies of 0.9182 on CIFAR-10, 0.6948 on CIFAR-100, and 0.7421 on ModelNet40. Held-out test recomputations give top-1 accuracies of 0.8995, 0.6963, and 0.6965, respectively.

What were the results of the analyses? The artifact set shows that the implementation trains successfully in both 2D and 3D modes and that the final checkpoints have non-trivial held-out performance. It also shows that the ModelNet40 long retraining run did not improve over its short-budget sweep result.

How did the analyses answer the central question? They support the implementation claim: the shifted-window transformer was made dimension-aware in code and can be trained through one reproducible pipeline on multiple spatial dimensionalities. The analyses do not support a state-of-the-art benchmark claim.

What does this answer tell us about the broader field? It suggests that architectural ideas such as shifted-window attention can be transferred beyond their original 2D setting when the surrounding software stack is designed around spatial tuples rather than image-specific assumptions.

1. Introduction

Vision transformers have changed how visual recognition models are designed, but early transformer architectures often scaled poorly with input resolution because self-attention was computed globally over all tokens. The original Swin Transformer addressed this issue by replacing global attention with a hierarchy of local windowed attention blocks and a shifted-window mechanism that lets neighboring windows exchange information without restoring full quadratic attention cost across the entire image (Liu et al., 2021). This design made transformer-based models much more practical for mainstream computer vision tasks and established Swin as one of the most influential hierarchical transformer backbones.

The limitation that motivated this project is that most implementations and much of the surrounding tooling remain strongly biased toward 2D imagery. In many research and engineering settings, however, the data are not naturally 2D. Medical scans, voxelized CAD objects, and other spatial measurements often live in three dimensions. Video and spatio-temporal signals can require even higher-dimensional reasoning. Reusing the shifted-window idea in these settings should be possible in principle, but real codebases often encode 2D assumptions in patch extraction, window bookkeeping, relative position indexing, data loading, and training orchestration. As a result, extending a 2D Swin implementation into a general N-dimensional system is less a matter of changing a single attention layer than of making the entire modeling and experimentation stack dimension-aware.

The goal of this practical work was therefore to build an N-dimensional shifted-window transformer in JAX and to verify that the resulting implementation can be trained in a realistic, traceable workflow. The emphasis of the project is not on proposing a new transformer variant. Instead, the contribution is an implementation and systems contribution: one codebase, one configuration-driven command surface, and one training stack that can support both 2D image classification and 3D voxel classification by changing configuration rather than rewriting architecture-specific code. This project also aimed to make the surrounding workflow credible from a research-practice perspective by supporting hyperparameter sweeps, queue-based job execution, logging, checkpointing, and dataset validation before a run starts.

The repository snapshot analyzed in this report contains exactly the kind of evidence needed for such a claim. It includes preserved sweeps and restored-best retraining checkpoints for CIFAR-10, CIFAR-100, and a voxelized ModelNet40 dataset. The final checkpoints reach validation top-1 accuracies of 0.9182, 0.6948, and 0.7421, respectively, with recomputed held-out test top-1 accuracies of 0.8995, 0.6963, and 0.6965. It also includes queue, tmux, and training logs showing how the workloads were orchestrated through the same CLI-based pipeline. These artifacts demonstrate that the project is not merely a collection of isolated modules, but a functioning experimentation framework.

This report is structured as follows. section 2 situates the implementation in the context of the original Swin paper, the JAX/Flax ecosystem, and the datasets used here. section 3 explains how the architecture and training system were generalized from fixed 2D assumptions to arbitrary spatial dimensionalities. section 4 documents the experimental setup, the datasets, the hardware, and the sweep procedures. section 5 interprets the validation-selected checkpoints and the recomputed test metrics, including the gap between sweep-best and final retraining performance for ModelNet40. Finally, section 6 summarizes the main constraints of the preserved evidence, especially the lack of repeated-seed uncertainty estimates. The report deliberately stays within those limits so

that every conclusion can be traced back to the saved artifacts rather than to assumptions or reconstructed claims.

2. Related Work

NDSwin-JAX builds directly on the Swin Transformer of Liu et al. (2021). Swin made vision transformers more practical by replacing global attention with local window attention and shifted windows. This retained a hierarchical feature pyramid while reducing the cost of attention on dense visual grids.

The same idea has already proved useful beyond ordinary 2D classification. Video Swin adapts shifted-window attention to video recognition by adding a temporal dimension while preserving the locality and hierarchy of the original model (Liu et al., 2022). Swin3D applies a Swin-style backbone to sparse 3D scene understanding, showing why local attention and relative position information are natural tools for large 3D inputs (Yang et al., 2023). These projects motivate the central engineering goal of this work: implement the spatial operators once, over tuples of dimensions, instead of maintaining separate 2D and 3D code paths.

The implementation is built in JAX and Flax, which makes this project partly a software-engineering exercise in reusable array programming rather than only a model-training exercise (Flax Team, 2024; JAX Authors, 2018). That matters for the report's central question: the evidence has to show not only that one checkpoint achieves a useful score, but that the code path remains configurable, inspectable, and reproducible across dimensionalities.

Medical imaging is especially relevant because many datasets are naturally volumetric. UNETR reformulates 3D medical segmentation as transformer-based sequence modeling with a U-shaped decoder (Hatamizadeh et al., 2022b). Swin UNETR then combines a hierarchical Swin encoder with medical segmentation decoders and self-supervised pre-training on large CT collections (Hatamizadeh et al., 2022a; Tang et al., 2022). nnFormer is another volumetric transformer that interleaves convolution and attention for 3D segmentation (Zhou et al., 2021). Together, these works show that high-dimensional transformer backbones are not just a software curiosity; they are a practical direction for volumetric data.

This report is narrower than those papers. It does not propose a new medical segmentation benchmark or claim state-of-the-art accuracy. Its contribution is an auditable JAX/Flax implementation of the shifted-window backbone that can be configured for different spatial dimensionalities, plus the training and queue machinery needed to run 2D and 3D experiments reproducibly.

3. Methods and Implementation

The central implementation task was to remove implicit 2D assumptions from the Swin pipeline while preserving the original architectural pattern. In the analyzed repository snapshot, the resulting model is exposed as `NDSwinTransformer` and configured through `num_dims`, `patch_size`, and `window_size`. The model still follows the familiar Swin structure: patch embedding, multiple hierarchical transformer stages, and a classifier head. The difference is that spatial operators now work on tuples of arbitrary length instead of fixed height–width pairs.

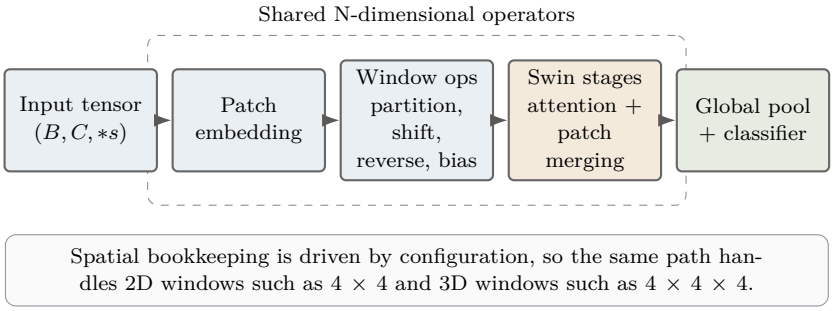

Figure 1: Overview of the N-dimensional shifted-window pipeline used in NDSwin-JAX. The grouped middle blocks are the generic spatial operators: they reuse the same code path for 2D image windows and 3D voxel windows by reading `num_dims`, `patch_size`, and `window_size` from configuration.

At the model level, patch embedding converts channel-first tensors of shape $(B, C, *\text{spatial})$ into token grids whose dimensionality depends only on the configured patch size. The generalized window operators then partition tensors of shape $(B, *\text{spatial}, C)$ into local windows, reverse that partitioning after attention, and apply cyclic shifts along each spatial axis. Relative position bias indexing is likewise computed from the configured window shape. These are exactly the components that fail first in a Swin implementation that assumes only two spatial coordinates. Replacing those assumptions with tuple-based shape arithmetic lets the same attention block process both 4×4 image windows and $4 \times 4 \times 4$ voxel windows.

The implementation also preserves the hierarchical structure of Swin. The model stacks multiple stages, each with its own depth, number of heads, and stochastic depth schedule. Between stages, patch merging reduces spatial resolution while increasing the embedding dimension. Because these transformations operate on generic spatial tuples, the same stage logic applies to the CIFAR-10, CIFAR-100, and ModelNet40 models analyzed here. In practice, moving from 2D to 3D becomes a configuration change rather than an architectural rewrite.

The project extends beyond the model definition into the training system. The repository's canonical interface is the `ndswin` CLI, which supports training, sweeps, auto-sweeps, queue execution, dataset export, and validation. This matters for reproducibility because the same command surface launches both the 2D and 3D workflows. In the preserved artifact set, auto-sweep and retraining workflows produce final checkpoints for CIFAR-10, CIFAR-100, and ModelNet40. The evidence therefore covers both the model and the orchestration layer expected in a practical research project.

Training itself is implemented with JAX-based optimization and data-parallel sharding. The trainer constructs a JAX mesh over the visible devices and shards batches along the batch dimension using `NamedSharding`. This is relevant because the preserved auto-sweep logs for CIFAR-100 and ModelNet40 show that the CLI selected four GPUs automatically rather than relying on a manually fixed device list. The code chooses only devices that satisfy memory and utilization thresholds, so the report does not overstate the hardware actually evidenced by the preserved logs.

Finally, the implementation includes guard rails around data and configuration. Dataset contracts are validated before training starts, the ModelNet40 export is described by a manifest containing split counts and class information, and the sweep and queue layers record summaries to JSON. These details matter scientifically because they make the

Table 1: Execution environment information preserved for the reported runs. The software constraints are grounded in `pyproject.toml`; runtime device selection is confirmed by the saved training logs. Host CPU and RAM inventory was not captured in the committed artifacts.

Item	Value
Hardware evidenced by logs	Four CUDA devices selected for the preserved CIFAR-100 and ModelNet40 final runs: GPU IDs $[1, 2, 3, 0]$
Runtime device policy	Automatic GPU selection through the CLI with minimum free memory, maximum used memory, and maximum utilization thresholds
Core software constraints	JAX 0.7.1, jaxlib 0.7.1; Flax, Optax, and Orbax checkpointing resolved from the project environment constraints

evidence auditable. The results reported later are extracted from structured artifacts that preserve configuration, runtime metadata, and metric histories rather than from informal notes or screenshots.

4. Experimental Setup

The experiments in this report are restricted to the final publishable artifact set preserved under `report/data/sources/`. This artifact set combines the repaired CIFAR-10 retraining run from May 18, 2026, the repaired CIFAR-100 sweep-and-retrain run from May 19–20, 2026, and the stabilized ModelNet40 voxel run from April 29, 2026. The goal is to document what was actually built, debugged, and measured, not to mix the strongest-looking numbers from unrelated runs.

The first workload is CIFAR-10, a 10-class image classification task with 32×32 RGB inputs (Krizhevsky, 2009). The second workload is CIFAR-100, which keeps the same input resolution but increases the label space to 100 classes. Both CIFAR loaders use a deterministic stratified validation split from the training set and preserve the official test split for post-training evaluation. The final CIFAR-10 and CIFAR-100 checkpoints were selected by validation accuracy and then recomputed on the held-out test split.

The third workload is a voxelized ModelNet40 export (Wu et al., 2015). The repository manifest describes 8,858 training samples, 985 train-derived validation samples, and 2,468 test samples, with one input channel and a spatial resolution of 32^3. This experiment uses `num_dims = 3`, which activates the generalized 3D path in patch embedding, window partitioning, shifting, and patch merging. A held-out test recomputation is preserved for the final restored checkpoint, but the report remains explicit that the model was selected by validation accuracy.

Hyperparameter search is performed through configuration-driven random search. The preserved CIFAR-10 sweep evaluated 48 trials with validation accuracy as the selection metric and a 60-epoch budget per trial; trials 3 and 33 tied at the best validation accuracy, and the preserved auto-best configuration uses the first tied trial. The fixed CIFAR-100 sweep evaluated 24 trials with a 50-epoch budget per trial. The ModelNet40 sweep

Table 2: Experimental ledger for the preserved runs. Sweep budgets and summed trial wall-clock times come from the committed sweep summaries; restored epochs and final wall-clock times come from the final `metrics.json` files.

Workload	**Splits used**	**Sweep budget/time**	**Final training state**	**Final wall-clock**
CIFAR-10	45k train / 5k validation / 10k test	48 trials, 60 epochs each; 32.83 h summed	600-epoch budget; stopped at 392; restored epoch 352	14,866.06 s (4.13 h)
CIFAR-100	45k train / 5k validation / 10k test	24 trials, 50 epochs each; 13.58 h summed	600-epoch budget; stopped at 350; restored epoch 300	12,043.98 s (3.35 h)
ModelNet40	8,858 train / 985 validation / 2,468 test	15 trials, 10 epochs each; 1.08 h summed	200-epoch budget; stopped at 27; restored epoch 7	990.99 s (0.28 h)

Table 3: Final retraining configurations. The table keeps the main effective knobs visible while the exact JSON and metric files remain preserved in Appendix A.

Workload	**Geometry and model**	**Optimization**	**Regularization**
CIFAR-10	2D; patch 2×2; window 4×4; embed 96; depths $[2, 2, 18, 2]$	batch 128; lr 4.300210×10^{-4}; wd 1.041082×10^{-2}; warmup 20	drop 0.05; drop path 0.1; cutmix 0.5; color jitter
CIFAR-100	2D; patch 2×2; window 4×4; embed 96; depths $[2, 2, 18, 2]$	batch 128; lr 2.047168×10^{-4}; wd 2.672492×10^{-2}; warmup 30	drop path 0.3; mixup 0.2; cutmix 0.5; cutout 16
ModelNet40	3D; patch $2 \times 2 \times 2$; window $4 \times 4 \times 4$; embed 72; depths $[2, 2, 6, 2]$	batch 64; lr 1.262274×10^{-3}; wd 7.049835×10^{-3}; warmup 3	drop path 0.1

evaluated 15 successful trials with a 10-epoch budget per trial. The sweep plots below show the selected trial for each workload.

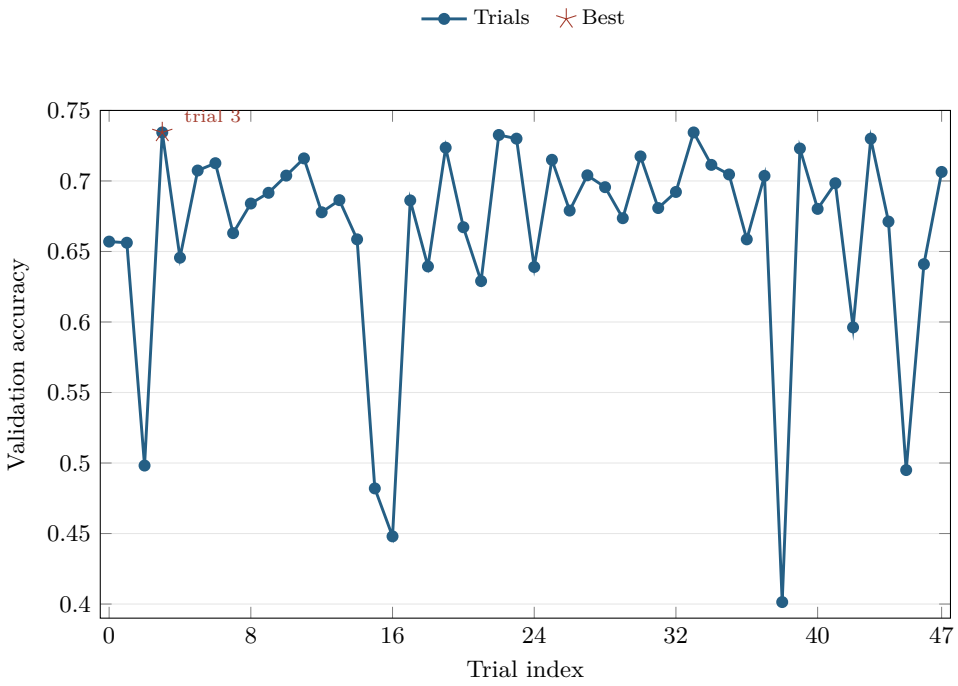

Figure 2: Validation accuracy across the 48 preserved CIFAR-10 sweep trials. The highlighted star marks trial 3, one of two tied best trials at 0.7344 under the 60-epoch sweep budget. The later repaired retraining run used this selected artifact as its base but lowered drop-path regularization.

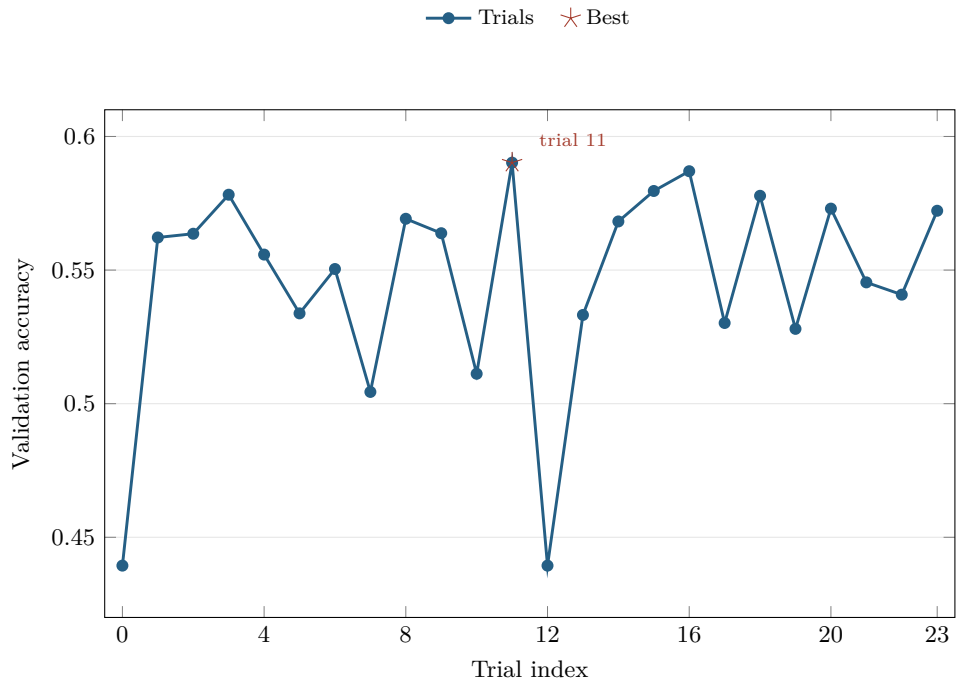

Figure 3: Validation accuracy across the 24 fixed CIFAR-100 sweep trials. The highlighted star marks trial 11, which reached 0.5902 under the 50-epoch sweep budget and was selected for full retraining.

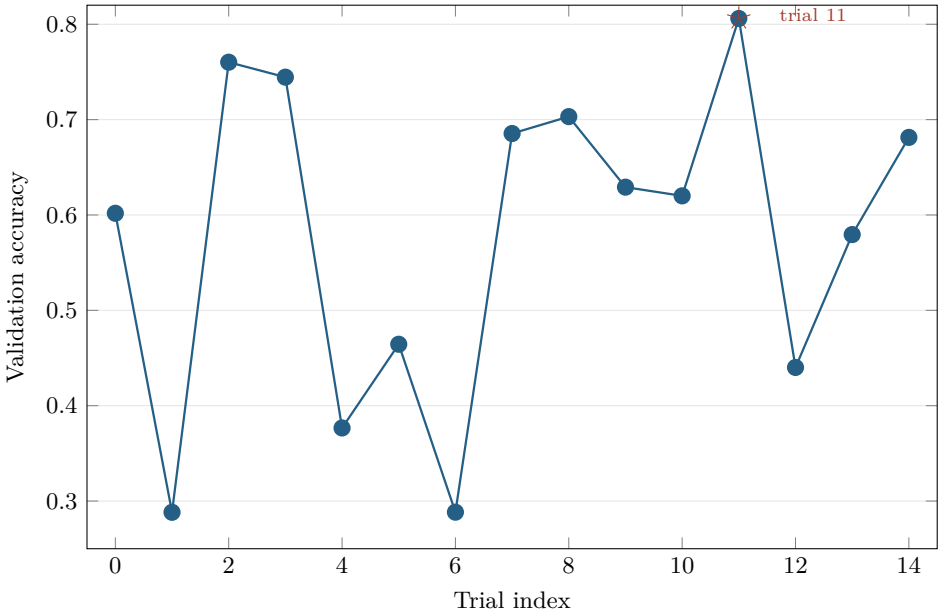

Figure 4: Validation accuracy across the 15 successful ModelNet40 sweep trials in the resumed stable run. The highlighted star marks trial 11, which reached 0.8061 under the 10-epoch sweep budget.

After sweep selection, each task is converted into the final retraining configuration summarized in Table 3. CIFAR-100 preserves the selected drop-path setting, while CIFAR-10 and ModelNet40 use the selected artifacts as bases and lower `drop_path_rate` for the long retraining runs. The restored best state, not the nominal epoch budget, determines the final reported checkpoint. Table 2 makes this explicit because the wall-clock time and restored epoch are part of the reproducibility evidence, not just implementation details.

The experiments were executed through the repository's CLI, sweep, queue, and tmux-backed workflow mechanisms, which are themselves part of the practical contribution. The high-level command form for the final sweep-plus-retrain jobs is `make auto-sweep-tmux SWEEP=configs/sweeps/<sweep>.yaml TRAIN_EPOCHS=<epochs>`; the detached queue wrapper uses `make benchmark-tmux` for the committed 2D-to-3D benchmark queue. The committed extraction inputs and the published Hugging Face weight repositories are listed in Appendix A.

5. Results and Interpretation

The most important result of this project is qualitative before it is quantitative: one codebase successfully trained on 2D image benchmarks and a 3D voxel benchmark with no architecture rewrite between the tasks. The same model family, training stack, and CLI workflow were reused across all workloads, with task-specific behavior determined by configuration. That is the core empirical validation of the N-dimensional implementation.

For CIFAR-10, the original preserved sweep produced a tied best validation accuracy of 0.7344 at trials 3 and 33 under the 60-epoch sweep budget; trial 3 is the preserved selected artifact used as the retraining base. After the data-loader randomness bug was fixed, long retraining with a 600-epoch budget and lower drop-path regularization produced a much stronger restored checkpoint: validation accuracy 0.9182, validation top-5 accuracy 0.9930, and validation loss 0.3237 at epoch 352. A held-out test recomputation of the restored checkpoint reached 0.8995 top-1 accuracy and 0.9936 top-5 accuracy. The training curves in Figure 5 show a stable long retraining run rather than a metric-only artifact.

For CIFAR-100, the fixed 24-trial sweep selected trial 11 with a validation accuracy of 0.5902 and top-5 validation accuracy of 0.8612 under the 50-epoch sweep budget. Full retraining improved the restored validation checkpoint to 0.6948 top-1 accuracy and 0.8856 top-5 accuracy at epoch 300. The held-out test recomputation is closely aligned with validation: 0.6963 top-1 accuracy, 0.8819 top-5 accuracy, and loss 1.3311. This result is not a state-of-the-art CIFAR-100 claim, but it is a useful sanity check that the repaired 2D pipeline learns a non-trivial 100-class task. The curve is shown in Figure 6.

For ModelNet40, the preserved sweep completed 15 successful trials, and the best sweep configuration, trial 11, achieved a validation accuracy of 0.8061 and a top-5 validation accuracy of 0.9499. Full retraining from that selected artifact, with a longer budget and lower drop-path regularization, did *not* improve on the sweep-best score; instead, the restored validation checkpoint reached 0.7421, with top-5 validation accuracy 0.9306 and validation loss 0.9640. A held-out test recomputation reached 0.6965 top-1 accuracy and 0.9149 top-5 accuracy. This does not imply that the architecture fails on 3D data. Rather, it shows that this particular long retraining schedule did not translate the sweep-best artifact into a stronger final checkpoint.

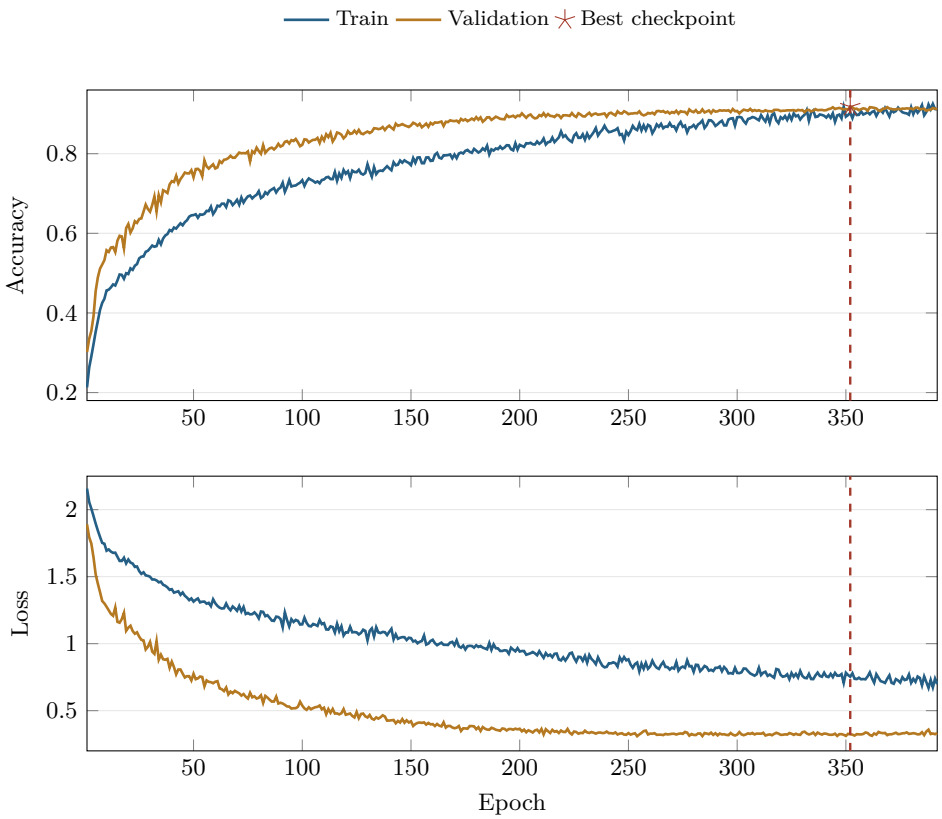

Figure 5: Training history for the final repaired CIFAR-10 retraining run. The dashed marker highlights the best restored checkpoint at epoch 352 with validation accuracy 0.9182; early stopping terminated the run at epoch 392.

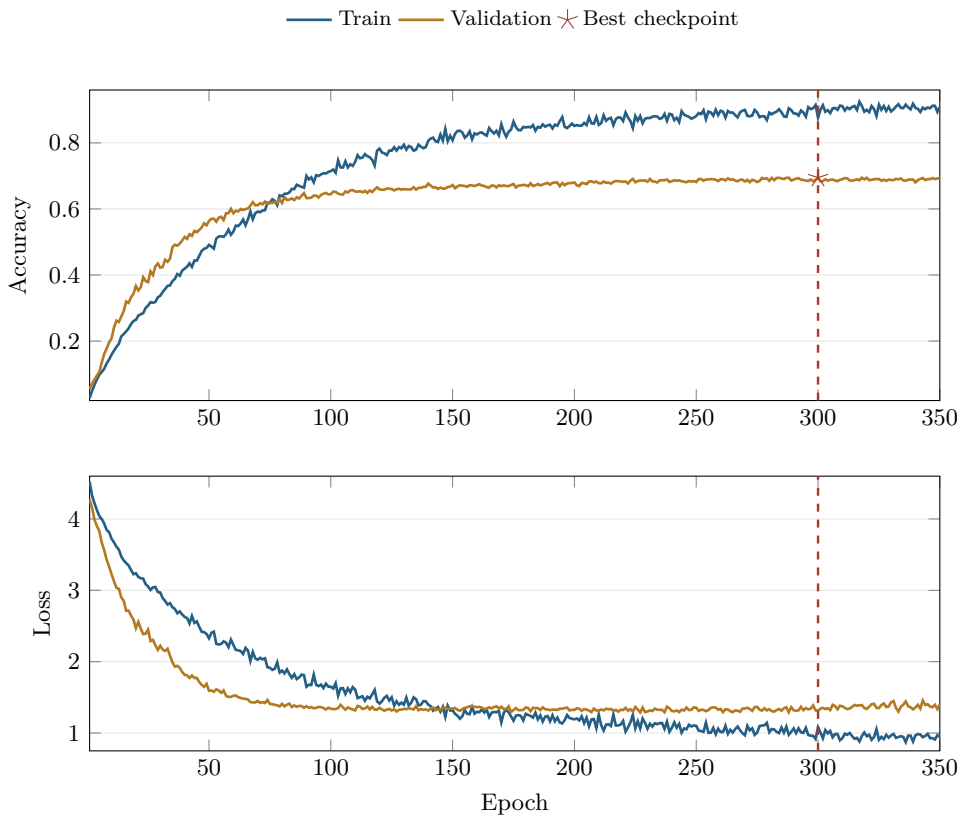

Figure 6: Training history for the final repaired CIFAR-100 retraining run. The dashed marker highlights the best restored checkpoint at epoch 300 with validation accuracy 0.6948; early stopping terminated the run at epoch 350.

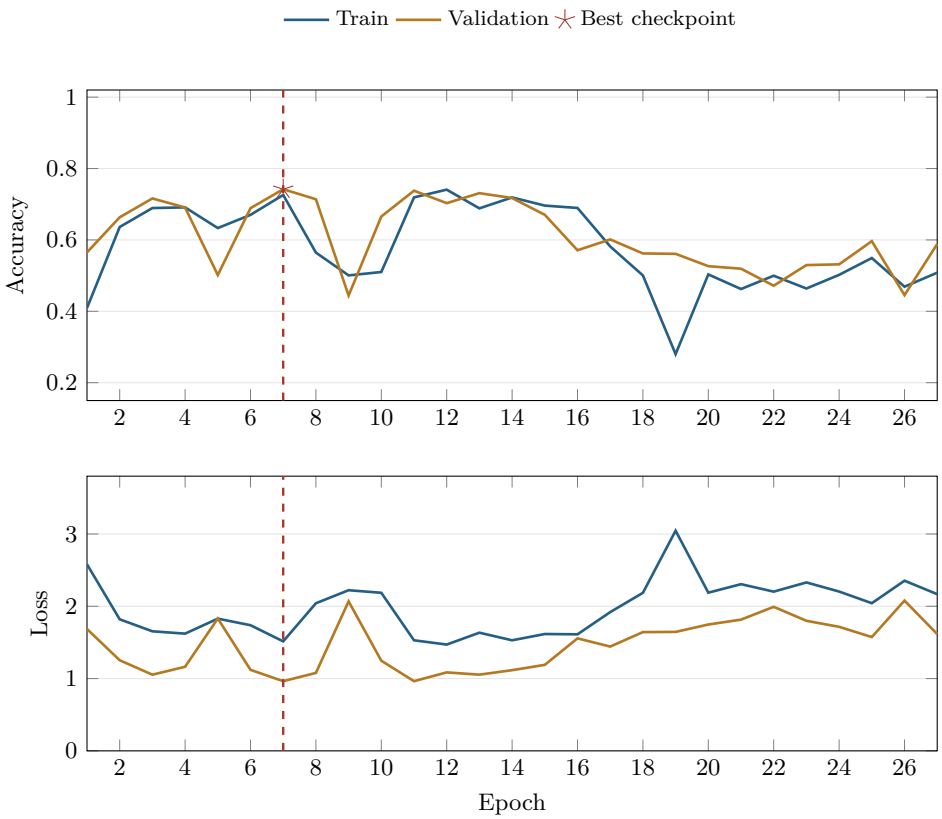

Figure 7: Training history for the final ModelNet40 retraining run. The dashed marker highlights the best restored checkpoint at epoch 7 with validation accuracy 0.7421; early stopping stopped the run at epoch 27.

Table 4: Summary of the preserved final results. Sweep metrics correspond to the best completed sweep trial; final validation and test metrics correspond to the restored checkpoint selected during full retraining. Delta columns report final validation minus sweep validation and test top-1 minus final validation.

Dataset	Sweep	Final	Δ	Test	Δ	Top-5	Params	Time	Best Ep.
CIFAR-10	0.7344	0.9182	+0.1838	0.8995	−0.0187	0.9936	48.8M	4.13 h	352
CIFAR-100	0.5902	0.6948	+0.1046	0.6963	+0.0015	0.8819	48.9M	3.35 h	300
ModelNet40	0.8061	0.7421	−0.0640	0.6965	−0.0455	0.9149	16.4M	0.28 h	7

The comparison between sweep-best and final restored-checkpoint performance is summarized in Table 4. The distinction between sweep-best, validation-selected final checkpoint, and recomputed held-out test metrics remains explicit, and the delta columns make the different retraining outcomes visible. No error bars are shown because the preserved evidence for this submission consists of single retained runs rather than repeated-seed experiments.

Overall, the results support a restrained but clear interpretation. NDSwin-JAX is not yet supported by the repeated, standardized benchmark campaign that would justify state-of-the-art claims. Nevertheless, the preserved final artifact set shows that the implementation is functional, configurable, and reusable across dimensionalities. The final checkpoints are published as Hugging Face model repositories for CIFAR-10 (https://huggingface.co/volodymyr-yelisieiev/ndswin-cifar10-final), CIFAR-100 (https://huggingface.co/volodymyr-yelisieiev/ndswin-cifar100-final), and ModelNet40 (https://huggingface.co/volodymyr-yelisieiev/ndswin-modelnet40-final); Appendix A lists these links together with the committed metric files from which the tables and plots are regenerated. The project succeeds as a practical work in applied machine learning engineering: it translates a known architecture into a broader implementation setting and validates that translation with credible, traceable experiments.

6. Limitations and Conclusion

Several limitations of the preserved evidence need to be stated explicitly. The most important is that all final checkpoints were selected by validation accuracy. Held-out test metrics were recomputed for the final CIFAR-10, CIFAR-100, and ModelNet40 checkpoints, but those test metrics were not used for model selection and are still single-run measurements. They should therefore not be compared casually to published Swin results reported under standardized multi-run or carefully tuned benchmark protocols.

The second major limitation is the absence of repeated-seed experiments, error bars, and statistical significance tests. The project supports reproducibility in the engineering sense through configuration files, saved metrics, queue logs, Hugging Face weight exports, and deterministic artifact paths. However, reproducibility in the stronger statistical sense would require rerunning the same configuration multiple times and quantifying variation across seeds or training stochasticity. Those measurements are not available in the preserved artifact set.

The third limitation concerns the gap between sweep-best and final-retrain performance. CIFAR-10 and CIFAR-100 improved substantially after full retraining, while ModelNet40 restored a weaker checkpoint than the best short-budget sweep trial. The final CIFAR-10 and ModelNet40 retraining configurations also lowered drop-path regularization relative to their sweep-selected artifacts, so the report treats the runs as comparable evidence for dimensional generality, not as a perfectly controlled benchmark suite. Better retraining policies, stronger schedule transfer from sweep to full run, or repeated validation of the selected artifact would strengthen the empirical story.

Another limitation is the scope of the empirical comparison itself. The report evaluates two small 2D image datasets and one voxelized 3D object-recognition dataset. This is enough to demonstrate dimensional generality, but not to claim broad superiority across datasets, tasks, or modalities. The ModelNet40 experiment also depends on a voxelized export pipeline rather than on the raw representation used in some other 3D learning pipelines. Its validation split is train-derived and documented by the preserved dataset manifest, so the present report treats the 3D result as evidence that the implementation works on volumetric data, not as a universal statement about 3D vision performance.

With those boundaries in place, the overall conclusion is positive. The project successfully implemented a reusable N-dimensional shifted-window transformer framework in JAX and demonstrated that it can be trained in a practical workflow on CIFAR-10, CIFAR-100, and ModelNet40 voxels. The architecture-level generalization is real in code,

not merely conceptual: patch embedding, window partitioning, shifting, relative position handling, and stage composition all operate on configurable spatial dimensionality. The surrounding experimentation framework is likewise functional, supporting sweeps, queue execution, checkpointing, structured metric export, and publishable model-weight bundles.

In that sense, the work fulfills the core objective of a practical AI project based on the original Swin paper. It translates a strong published idea into a broader implementation setting, exposes the engineering decisions needed to make that translation work, and produces valid empirical evidence that the resulting system can learn on multiple data modalities. Future work should prioritize repeated-seed uncertainty estimates, stronger retraining policies after sweep selection, direct comparison to published Swin baselines, and larger-scale datasets where transformer architectures are known to be more competitive. Those additions would strengthen the scientific completeness of the evidence, but they are not required to support the central conclusion of this report: NDSwin-JAX is a credible, traceable N-dimensional implementation of shifted-window transformers in JAX.

References

Flax Team (2024). *Flax: A neural network library and ecosystem for JAX*. https://github.com/google/flax. Accessed 2026-03-29.

Hatamizadeh, Ali, Vishwesh Nath, Yucheng Tang, Dong Yang, Holger Roth, and Daguang Xu (2022a). "Swin UNETR: Swin Transformers for Semantic Segmentation of Brain Tumors in MRI Images". In: *arXiv preprint arXiv:2201.01266*.

Hatamizadeh, Ali, Yucheng Tang, Vishwesh Nath, Dong Yang, Andriy Myronenko, Bennett Landman, Holger Roth, and Daguang Xu (2022b). "UNETR: Transformers for 3D Medical Image Segmentation". In: *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pp. 1748–1758.

JAX Authors (2018). *JAX: composable transformations of Python+NumPy programs*. https://github.com/google/jax. Accessed 2026-03-29.

Krizhevsky, Alex (2009). *Learning Multiple Layers of Features from Tiny Images*. Tech. rep. University of Toronto.

Liu, Ze, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo (2021). "Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows". In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 10012–10022. DOI: 10.1109/ICCV48922.2021.00986.

Liu, Ze, Jia Ning, Yue Cao, Yixuan Wei, Zheng Zhang, Stephen Lin, and Han Hu (2022). "Video Swin Transformer". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 3202–3211.

Tang, Yucheng, Dong Yang, Wenqi Li, Holger Roth, Bennett Landman, Daguang Xu, Vishwesh Nath, and Ali Hatamizadeh (2022). "Self-Supervised Pre-Training of Swin Transformers for 3D Medical Image Analysis". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 20730–20740.

Wu, Zhirong, Shuran Song, Aditya Khosla, Fisher Yu, Linguang Zhang, Xiaoou Tang, and Jianxiong Xiao (2015). "3D ShapeNets: A Deep Representation for Volumetric Shapes". In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 1912–1920. DOI: 10.1109/CVPR.2015.7298801.

Yang, Yu-Qi, Yu-Xiao Guo, Jian-Yu Xiong, Yang Liu, Hao Pan, Peng-Shuai Wang, Xin Tong, and Baining Guo (2023). "Swin3D: A Pretrained Transformer Backbone for 3D Indoor Scene Understanding". In: *arXiv preprint arXiv:2304.06906*.

Zhou, Hong-Yu, Jiansen Guo, Yinghao Zhang, Lequan Yu, Liansheng Wang, and Yizhou Yu (2021). “nnFormer: Interleaved Transformer for Volumetric Segmentation”. In: *arXiv preprint arXiv:2109.03201*.

Appendix A. Committed Report Evidence Bundle

The public code for this project is available at https://github.com/volodymyr-yelisieiev/ndswin-jax. The quantitative evidence in this report is tied to the committed report evidence bundle under `report/data/sources/`. The derived plot tables in `report/data/*.csv` are regenerated from those local copies by `report/extract_report_data.py`. The restored-best checkpoints were also packaged as Hugging Face model repositories so the final weights can be inspected independently of the local training machine.

The figures and tables in this report are derived from the following committed bundle entries. Path entries in the table are relative to `report/data/sources/`: for example, `logs/...` denotes the committed copy under `report/data/sources/logs/...`, not a live root-level `logs/` directory from the training server. The repository's root-level runtime directories such as `logs/`, `outputs/`, `data/`, and `configs/auto_best/` are intentionally ignored by Git and are not required to build or audit this report. Final metric files contain the effective training configurations used for the reported checkpoints. The sweep-selected config files are included as traceability artifacts, but some full retraining runs intentionally changed runtime fields such as epoch budget after sweep selection. The table also lists the external Hugging Face repositories that host the exported final weights.

Resource	Bundle-relative path or external repository
Sweep summaries	`outputs/sweeps/cifar10_tuned_hyperparam_sweep/summary.json`
	`outputs/sweeps/cifar100_tuned_hyperparam_sweep_fixed_20260519/summary.json`
	`outputs/sweeps/modelnet40_stable_hyperparam_sweep/summary.json`
Final metrics	`outputs/cifar10/cifar10_rngfix_retrain600_20260518/checkpoints/metrics.json`
	`outputs/cifar100/cifar100_tuned_0fd87157_20260520_112244/checkpoints/metrics.json`
	`outputs/volume_folder/modelnet40_f67c9398_20260429_165927/checkpoints/metrics.json`
Test metrics	`outputs/cifar10/cifar10_rngfix_retrain600_20260518/checkpoints/test_metrics.json`
	`outputs/cifar100/cifar100_tuned_0fd87157_20260520_112244/checkpoints/test_metrics.json`
	`outputs/volume_folder/modelnet40_f67c9398_20260429_165927/checkpoints/test_metrics.json`
Logs	`logs/tmux/auto_sweep/autosweep_20260519_214743.log`
	`logs/tmux/auto_sweep/autosweep_20260429_155416.log`
	`logs/cifar10/cifar10_rngfix_retrain600_20260518.log`
	`logs/cifar100/cifar100_tuned_0fd87157_20260520_112244.log`
	`logs/volume_folder/modelnet40_f67c9398_20260429_165927.log`
Dataset manifest	`data/modelnet40/dataset_manifest.json`
Sweep-selected configs	`configs/auto_best/best_cifar10_cifar10_tuned_05f7e1b8_20260427_234823_trial003.json`
	`configs/auto_best/best_cifar100_cifar100_tuned_a9ef3d19_20260520_040951_trial011.json`
	`configs/auto_best/best_volume_folder_modelnet40_e548c891_20260429_164145_trial011.json`
Published weights	https://huggingface.co/volodymyr-yelisieiev/ndswin-cifar10-final
	https://huggingface.co/volodymyr-yelisieiev/ndswin-cifar100-final
	https://huggingface.co/volodymyr-yelisieiev/ndswin-modelnet40-final

Code-level implementation claims are grounded in the tracked repository files `pyproject.toml`, `src/ndswin/core/window_ops.py`, `src/ndswin/core/patch_embed.py`, `src/ndswin/core/attention.py`, `src/ndswin/models/swin.py`, `src/ndswin/training/trainer.py`, and `src/ndswin/utils/gpu.py`.